Forward pass and Backward propagation with vanilla RNN

```
import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(suppress=True)
def sigmoid(x):
    return 1/(1+np.exp(-x))
def softmax(x):
    exp_x = np.exp(x - np.max(x))
    return exp_x/np.sum(exp_x)
def cross_entropy(y_true,y_pred):
    return -np.sum(y_true * np.log(y_pred + 1e-12))
input_size = 3
hidden_size = 2
output_size = 3
def init_gate():
    return np.random.random(hidden_size, input_size +
                            hidden_size) * 0.1
wf = init_gate()
wi = init_gate()
wo = init_gate()
wc = init_gate()
bf = np.zeros((hidden_size,1))
bi = np.zeros((hidden_size,1))
bo = np.zeros((hidden_size,1))
bc = np.zeros((hidden_size,1))
Why = np.random.randn(output_size, hidden_size) * 0.1
by = np.zeros((output_size,1))
x1 = np.array([[1],[0],[0]])
x2 = np.array([[0],[1],[0]])
x3 = np.array([[0],[0],[1]])
inputs = [x1, x2, x3]
t1 = np.array([[0],[1],[0]])
t2 = np.array([[0],[0],[1]])
t3 = np.array([[1],[0],[0]])
targets = [t1, t2, t3]
```

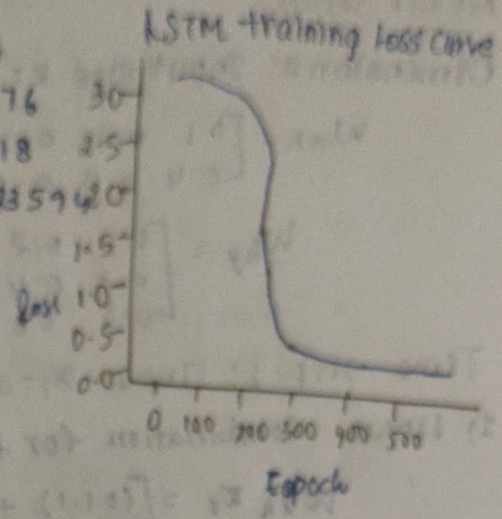
```
L-r = 0.1
threshold = 0.05
max-epochs = 1000
clip-value = 5
def clip-gradients(grad, clip-value):
    return np.clip(grad, -clip-value, clip-value)
Loss-history = []
for epoch in range(max-epochs)
    h = np.zeros((hidden-size, 1))
    c = np.zeros((hidden-size, 1))
    hs, cs, zs = [h], [c], []
    fs, ins, os, c-tildes = [], [], [], []
    ps = []
    loss = 0
    for t in range(3):
        z = np.vstack((h, inputs[t]))
        zs.append(z)
        f = sigmoid(Wf @ z + bf)
        i = sigmoid(Wi @ z + bi)
        o = sigmoid(Wo @ z + bo)
        c-tilde = np.tanh(Wc @ z + bc)
        c = f * c + i * c-tilde
        h = o * np.tanh(c)
        y = Why @ h - by
        p = softmax(y)
        fs.append(f)
        ins.append(i)
        os.append(o)
        c-tildes.append(c-tilde)
        hs.append(h)
        cs.append(c)
        ps.append(p)
        loss += cross-entropy(targets[t], p)
Loss-history.append(loss)
if loss < threshold:
    print("Training converged at epoch", epoch+1)
    break
```

```
dwf = np.zeros_like(wf)
dwi = np.zeros_like(wi)
dwo = np.zeros_like(wo)
dwc = np.zeros_like(wc)
dbf = np.zeros_like(bf)
dbi = np.zeros_like(bi)
dbo = np.zeros_like(bo)
dbc = np.zeros_like(bc)
dWhy = np.zeros_like(Why)
dby = np.zeros_like(by)
dh_next = np.zeros((hidden_size,1))
dc_next = np.zeros((hidden_size,1))
for t in reversed(range(3)):
    dy = ps[t] - targets[t]
    dWhy += dy @ hs[t+1].T
    dby += dy
    dh = Why.T @ dy + dh_next
    dc = dh * os[t] * (1 - np.tanh(cs[t+1])**2) + dc_next
    do = dh * np.tanh(cs[t+1])
    do_raw = do * os[t] * (1 - os[t])
    df = dc * cs[t]
    df_raw = df * fs[t] * (1 - fs[t])
    di = dc * c_tildes[t]
    di_raw = di * ins[t] * (1 - ins[t])
    dc_tilde = dc * ins[t]
    dc_tilde_raw = dc_tilde * (1 - c_tildes[t] ** 2)
    dwf += df_raw @ zs[t].T
    dwi += di_raw @ zs[t].T
    dwo += do_raw @ zs[t].T
    dwc += dc_tilde_raw @ zs[t].T
    dbf += df_raw
    dbi += di_raw
    dbo += do_raw
    dbc += dc_tilde_raw
    dz = (wf.T @ df_raw + wi.T @ di_raw + wo.T @ do_raw
          + wc.T @ dc_tilde_raw)
    dh_next = dz[:hidden_size, :]
    dc_next = dc * fs[t]
for grad in [dwf, dwi, dwo, dwc, dWhy, dbf, dbi, dbo, dbc,
             dby]:
```

```
np.clip(grad, -clip_value, clip_value, out=grad)
    wf -= l_r * dwf
    wi -= l_r * dwi
    wo -= l_r * dwo
    wc -= l_r * dwc
    why -= l_r * dwhy
    bf -= l_r * dbf
    bi -= l_r * dbi
    bo -= l_r * dbo
    bc -= l_r * dbc
    by -= l_r * dby
    if (epoch+1) % 100 == 0:
        print("Epoch:", epoch+1, "Loss:", loss)
print("\nFinal loss:", loss)
print("\nFinal Predictions:")
for t in range(3):
    print(np.round(ps[t],2))
print("\n Final LSTM Weights and Biases:\n")
print("wf (Forget Gate weights):\n", np.round(wf,4))
print("bf (Forget Gate Bias):\n", np.round(bf,4))
print("\nwi (Input Gate weights):\n", np.round(wi,4))
print(" bi (Input Gate Bias):\n", np.round(bi,4))
print("\nwo (output Gate weights):\n", np.round(wo,4))
print("bo (output Gate Bias):\n", np.round(bo,4))
print("\nwc (candidate weights):\n", np.round(wc,4))
print("bc (candidate Bias):\n", np.round(bc,4))
print("\nwhy (output layer weights):\n", np.round(why,4))
print("by (output layer Bias):\n", np.round(by,4))
plt.plot(loss_history, color='blue')
plt.title("LSTM Training Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

Output

Epoch : 100 Loss: 2.9196 7770 9150656
Epoch : 200 Loss: 0.462142 568 767213
Epoch : 300 Loss: 0.1428683 2 54818576
Epoch : 400 Loss: 0.07351 60015147 1918
Epoch : 500 Loss: 0.0502 8962 4444 7359 4
Final Loss: 0.0499 7083 4 2708 48995
Final Predictions:
[[0.01] [0.99] [0.]]
[[0.01] [0.] [0.98]]
[[0.98] [0.01] [0.01]]



Final LSTM weights and Biases:

W_f (forget gate weights):
[[-0.4728, 0.2265 0.2574 -0.5708 0.3908]
[0.0425 -0.2335 0.1808 -0.5438 -0.6779]]

b_f (forget gate Bias):
[[-0.153] [-1.2373]]

W_i (input gate weights):
[[-0.0058 -0.1029 0.6103 0.7802 0.3269]
[0.0984 0.1708 0.2238 0.9789 0.7179]]

b_i (input gate bias):
[[1.6702] [1.6813]]

W_o (output Gate weights):
[[-0.0262 0.2419 0.8283 0.4517 -0.5152]
[-0.0574 0.0233 0.2486 1.0553 0.6112]]

b_o (output bias Gate):
[[1.9623] [1.9583]]

W_c (candidate weights):
[[0.1775 -0.0966 2.0175 -1.4481 -0.9436]
[0.5808 -0.5834 -0.5133 1.6442 1.4746]]

b_c (candidate Bias):
[[-0.3376] [0.048]]

W_{hy} (output Layer weights):
[[-2.4553 -2.6505] [3.9007 -1.4095] [-1.3353 3.9366]]

b_y (output Layer Bias): [[-0.4295] [0.3892] [0.0402]]